Projeto da Disciplina: Fundamentos de Machine Learning com Scikit-Learn

Aluna: Ingrid Ornelas Costa Santiago Gomes

Repositório do Projeto: GitHub

1. O Problema de Negócio e os Dados

O Desafio

Instituições financeiras liberam diariamente com o desafio de assumir lacra de maus pagadores antes de concisder crédito. Erros nessa decisão custa caro. Abordagens antigas, baseadas em regras fixas, já não dão conta de analisar a complexidade do comportamento humano. E aqui que a Inteligência Artificial entra: precisamos de modelos capazes de entender padrões sutis no histórico de clientes para prever se ele se vai não pagar a próxima fatura.

Nossa Base de Dados

- **Origem:** Utilizamos a base pública chamada default-of-credit-card-clients-dataset do UCI via Kaggle.
- **Tamanho:** 30.000 clientes e 24 features (variáveis como idade, escolaridade e histórico detalhado de faturas passadas).
- **Nossa Alvo:** A variável default.payment.next.month (0 = ele dar certo, 1 = ele pagar em dia).
- **O Problema Técnico:** É uma tarefa clássica de Classificação Binária, onde o peso de errar para um lado (aprovar um inadimplente) é muito pior do que errar para o outro.

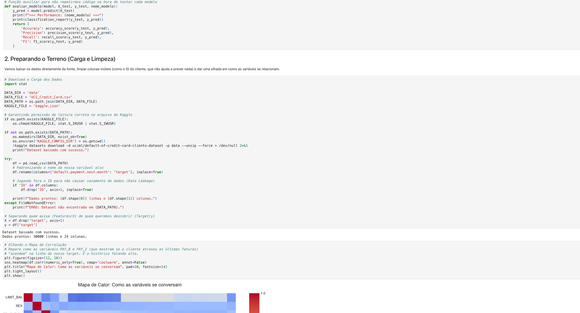
In [1]:
```
# Configuração inicial e verificação de dependências
import os
import sys

# Verificar se os pacotes estão instalados
if os.environ.get('requirements_installed', '0') != '1':
    print('Instalando requirements.txt...')
    !pip install -q -r requirements.txt
    os.environ['requirements_installed'] = '1'
    print('Instalação concluída!')
else:
    print('Dependências já instaladas.')

# Importação das bibliotecas
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    ConfusionMatrixDisplay
)

from sklearn.linear_model import Perceptron
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier

# Configurações para garantir que os resultados sejam sempre os mesmos (reprodutibilidade)
RANDOM_STATE = 42
N_JOBS = -1
%matplotlib inline

# Estilo para os gráficos
plt.style.use('seaborn-v0_8-whitegrid')

# Função auxiliar para não repetirmos código na hora de testar cada modelo
def avaliar_modelo(model, X_test, y_test, nome_modelo):
    y_pred = model.predict(X_test)
    print(f"=== Performance: {nome_modelo} ===")
    print(classification_report(y_test, y_pred))
    return {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1': f1_score(y_test, y_pred)
    }
```

2. Preparando o Terreno (Carga e Limpeza)

Vamos baixar os dados diretamente da fonte, limpar colunas inúteis (como o ID do cliente, que não ajuda a prever nada) e dar uma olhada em como as variáveis se relacionam.

In [2]:
```
# Download e Carga dos Dados
import os
import shutil

DATA_DIR = 'data'
DATA_PATH = 'data/credit_cards/'
DATA_FILE = os.path.join(DATA_PATH, DATA_FILE)
KAGGLE_FILE = 'kaggle.json'

# Garantindo permissão de leitura correta no arquivo do Kaggle
if os.path.exists(KAGGLE_FILE):
    os.chmod(KAGGLE_FILE, stat.S_IRUSR | stat.S_IWUSR)

if not os.path.exists(DATA_PATH):
    print("Baixando dataset do Kaggle...")
    os.makedirs('kaggle', exist_ok=True)
    !kaggle datasets download -d uci/default-of-credit-card-clients-dataset -p data --unzip --force > /dev/null 2>&1
    print("Dataset baixado com sucesso.")

try:
    df = pd.read_csv(DATA_PATH)
    # Padronizando o nome da nossa variável alvo
    df.rename(columns={'default.payment.next.month': 'target'}, inplace=True)

    # Removendo o ID, não faz sentido levar usuários de dados (data leakage)
    if 'ID' in df.columns:
        df.drop('ID', axis=1, inplace=True)

    print("Tamanho primeiro: {df.shape[0] linhas e {df.shape[1] colunas.")
except FileNotFoundError:
    print("ERRO: Dataset não encontrado em {DATA_PATH}.")

# Separando quem avisa (Features/X) de quem queremos descobrir (Target/y)
X = df.drop('target', axis=1)
y = df['target']

Dataset baixado com sucesso.
Dados prontos: 30000 linhas e 24 colunas.
```

In [3]:
```
# Olhando o Mapa de Correlação
# Vamos ver quais variáveis andam juntas e o PAY_2 (que mostra se o cliente atrasou as últimas faturas)
# "puxador" na linha de nossa target. É o histórico recente dele.
plt.figure(figsize=(12, 10))
corr_matrix = df.corrwith(df['target'], axis=1)
plt.title('Mapa de Calor: Como as variáveis se conversam', pad=20, fontsize=14)
plt.tight_layout()
plt.show()
```



3. O Baseline: Começando de forma simples (Perceptron)

Antes de usar árvores, vamos usar um antitiquo: O Perceptron é um modelo linear básico. A ideia aqui é responder à pergunta: "Dá pra separar bons e maus pagadores simplesmente traçando uma linha reta nos dados?"

Ajuste Técnico: Dados reais de crédito são bagunçados. Se colarmos o Perceptron puro aqui, ele vai ficar em loop infinito tentando achar uma separação perfeita que não existe. Para evitar isso (e as famosas warnings de não-convergência), ativamos o `shuffle=True`. Basicamente, estamos dizendo ao modelo: "Tente aprender, mas se você perceber que parou de evoluir, pode parar e entregar o que tem".

In [4]:
```
# Separando dados de treino e teste (usando a proporção de calotes com o stratify)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=RANDOM_STATE, stratify=y
)

# Modelos lineares precisam que os dados estejam na mesma escala (padronização)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Treinando o modelo Baseline
perc = Perceptron(
    eta0=0.1,
    shuffle=True,
    random_state=RANDOM_STATE
)

perc.fit(X_train_scaled, y_train)

# Avaliando o resultado para compararmos depois
resultado_finais = {}
resultado_finais['Perceptron'] = avaliar_modelo(perc, X_test_scaled, y_test, "Perceptron (Linear)")
```

```
=== Performance: Perceptron (Linear) ===
              precision    recall  f1-score   support

           0       0.80      0.97      0.76      7000
           1       0.36      0.06      0.10      2000

    accuracy                           0.63      9000
   macro avg       0.58      0.51      0.38      9000
weighted avg       0.73      0.63      0.48      9000
```

O que o Baseline nos ensina?

Olhando o relatório acima, o F1-Score para a classe 1 (inadimplentes) foi bem ruim. O modelo sofreu de underfitting (fico simples demais para a tarefa). Isso responde nossa pergunta inicial: **não, não dá para separar risco de crédito com uma linha reta.** O comportamento financeiro é complexo.

Mesmo assim, ele mentiu. Ele chamou no geral que até dea certo (cerca de 63% de acertos), ele todo o sentido do perfil de risco do negócio: arrasou recente (`PAY_2`, `PAY_3`) em uma enorme e positiva, ou seja, eles empurram a decisão de modelo para o lado da inadimplência.

In [5]:
```
coefs = pd.DataFrame(perc.coef_T, index=X.columns, columns=['Peso_Linear'])
print("Peso da variável que mais pesarem na decisão do Perceptron:")
print(coefs.abs().sort_values(by='Peso_Linear', ascending=False).head(5))
```

```
Peso da variável que mais pesarem na decisão do Perceptron:
     Peso_Linear
BILL_AMT1   0.911
PAY_AMT1    0.825
PAY_AMT2    0.804
PAY_4       0.600
PAY_5       0.416
```

4. Subindo o Nível: Árvore de Decisão

Já que uma linha reta não funciona, vamos usar uma Árvore de Decisão. Ela consegue falar no dados usando regras lógicas (aninhdas), bilando muito melhor com essa não-linearidade.

In [6]:
```
# Treinando uma Árvore sem limites de crescimento
dt_default = DecisionTreeClassifier(random_state=RANDOM_STATE)
dt_default.fit(X_train, y_train)

resultado_finais['DT Default'] = avaliar_modelo(dt_default, X_test, y_test, "Árvore (Sem Poda)")

print("\nProfundidade que a árvore atingiu: {dt_default.get_depth()}")
print("Quantas folhas (regras) ela criou: {dt_default.get_n_leaves()}")
```

```
=== Performance: Árvore (Sem Poda) ===
              precision    recall  f1-score   support

           0       0.83      0.81      0.82      7000
           1       0.38      0.40      0.39      2000

    accuracy                           0.72      9000
   macro avg       0.60      0.61      0.61      9000
weighted avg       0.73      0.72      0.72      9000

Profundidade que a árvore atingiu: 48
Quantas folhas (regras) ela criou: 3253
```

O perigo de decorar os dados (Overfitting)

Veja a profundidade enorme que a árvore acima atingiu. Ela criou milhares de regrinhas minuciosas. O que acontece aqui foi um clássico overfitting: a árvore "decorou" os clientes de treino, mas na hora foi ruim em um cliente novo.

Para resolver isso, vamos usar Validação Cruzada (`GridSearchCV`). Vamos testar várias combinações de "podas" (limitar a profundidade e o tamanho das folhas) e descobrir qual versão da árvore generaliza melhor, sem decorar nada.

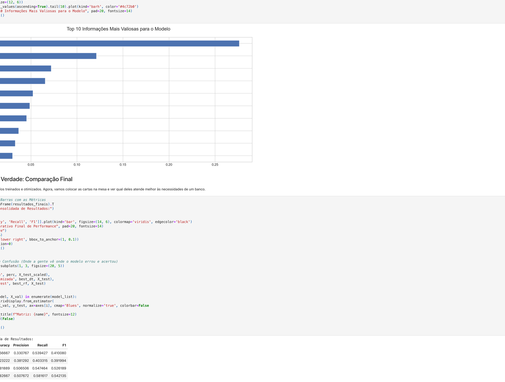
In [7]:
```
param_grid_dt = {
    'max_depth': [3, 4, 5, 6, 10],
    'min_samples_leaf': [10, 20, 50],
    'class_weight': [None, 'balanced']
}

grid_dt = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=RANDOM_STATE),
    param_grid=param_grid_dt,
    cv=5,
    scoring='f1',
    n_jobs=N_JOBS
)

grid_dt.fit(X_train, y_train)

best_dt = grid_dt.best_estimator_
resultado_finais['DT Otimizada'] = avaliar_modelo(best_dt, X_test, y_test, "Árvore Otimizada")

print("\nO melhor combinação encontrada foi: {grid_dt.best_params_}")
```

```
=== Performance: Árvore Otimizada ===
              precision    recall  f1-score   support

           0       0.87      0.85      0.86      7000
           1       0.51      0.55      0.53      2000

    accuracy                           0.78      9000
   macro avg       0.69      0.70      0.70      9000
weighted avg       0.79      0.78      0.78      9000

O melhor combinação encontrada foi: {'class_weight': 'balanced', 'max_depth': 5, 'min_samples_leaf': 50}
```

As Regras do Jogo

Quando limitamos o crescimento da árvore, ela se torna muito inteligente (97 para o negócio. Podemos literalmente imprimir as regras que ela aprendeu e mostrar para a diretoria ou para um auditor. Veja o desenho abaixo: ela valida primeiro o histórico de pagamento (`PAY_0`) e depois verifica limites e saldos.

In [8]:
```
plt.figure(figsize=(2, 8))
plot_tree(best_dt, feature_names=list(X.columns), class_names=['Pago', 'Default'], filled=True, max_depth=2, fontsize=11, rounded=True)
plt.title('A Lógica da Decisão da Nossa Árvore (Top 3 Níveis)', pad=20, fontsize=14)
plt.tight_layout()
plt.show()
```



5. O Peso Pesado: Random Forest (Ensemble)

Uma árvore é ótima porque é explicável, mas ela é um pouco instável. Ela mudaremos um pouquinho os dados, a árvore muda inteira.

A solução? Plantar uma floresta. O **Random Forest** cria dezenas de árvores diferentes e tira a média das decisões delas. E um modelo muito mais robusto e assertivo. Assim como floresta antes, vamos garantir a qualidade usando Validação Cruzada.

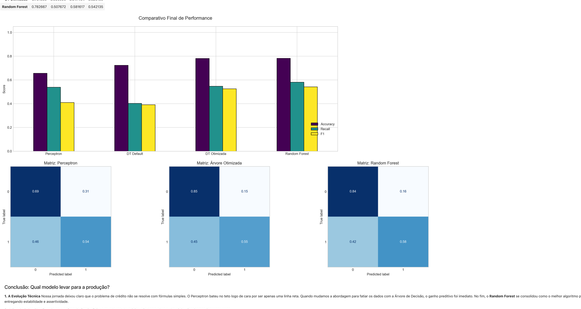
In [9]:
```
param_grid_rf = {
    'class_weight': [None, 'balanced'],
    'max_depth': [8, 12, 15],
    'min_samples_split': [5, 10],
    'n_estimators': [50, 100]
}

print("Treinando nossa floresta e validando hiperparâmetros...")
grid_rf = GridSearchCV(
    estimator=RandomForestClassifier(random_state=RANDOM_STATE),
    param_grid=param_grid_rf,
    cv=3,
    scoring='f1',
    n_jobs=N_JOBS
)

grid_rf.fit(X_train, y_train)

best_rf = grid_rf.best_estimator_
resultado_finais['Random Forest'] = avaliar_modelo(best_rf, X_test, y_test, "Random Forest")

print("\nOs melhores parâmetros para a Floresta foram: {grid_rf.best_params_}")
```

```
Treinando nossa floresta e validando hiperparâmetros...
=== Performance: Random Forest ===
              precision    recall  f1-score   support

           0       0.88      0.93      0.90      7000
           1       0.55      0.38      0.44      2000

    accuracy                           0.78      9000
   macro avg       0.69      0.65      0.67      9000
weighted avg       0.76      0.78      0.76      9000

Os melhores parâmetros para a Floresta foram: {'class_weight': 'balanced', 'max_depth': 8, 'min_samples_split': 10, 'n_estimators': 50}
```

O que mais importa?

Não esqueçam: diferente das árvores de árvores de Floresta, mas podemos perguntar e ela quais variáveis foram mais cheiro para purificar as decisões. O gráfico abaixo mostra o "Top 10" do que realmente importa na hora de conceder crédito segundo o Random Forest.

In [10]:
```
importances = pd.Series(best_rf.feature_importances_, index=X.columns)
plt.figure(figsize=(12, 6))
importance.nlargest(10).sort_values(ascending=True).plot(kind='barh', color='#ec7014')
plt.title('Top 10 Informações Mais Valiosas para o Modelo', pad=20, fontsize=14)
plt.tight_layout()
plt.show()
```



6. A Hora da Verdade: Comparação Final

Temos nossos modelos treinados e eliminados. Agora, vamos colocar as cartas na mesa e ver qual deles aprende melhor do necessidades de um banco.

In [11]:
```
# 1. Gráfico de Barras com as Métricas
df_res = pd.DataFrame(resultados_finais).T
print("Tabela Consolidada de Resultados:")
display(df_res)

df_res[['Accuracy', 'Recall', 'F1']].plot(kind='bar', figsize=(14, 6), colormap='viridis', edgecolor='black')
plt.title('Comparativo Final de Performance', pad=20, fontsize=14)
plt.ylabel('Score')
plt.ylim(0.0, 1.1)
plt.legend(loc='lower right', bbox_to_anchor=(1, 0.1))
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# 2. Matrizes de Confusão (Onde a gente vê onde o modelo errou e acertou)
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
modelos = [
    ('Perceptron', perc, X_test_scaled),
    ('Árvore Otimizada', best_dt, X_test),
    ('Random Forest', best_rf, X_test)
]

for i, (name, model, X_set) in enumerate(modelos):
    ConfusionMatrixDisplay.from_estimator(
        model, X_set, y_test, ax=axes[i], cmap='Blues', normalize='true', colorbar=False
    )
    axes[i].set_title('Matriz: {name}', fontsize=12)
    axes[i].grid(False)

plt.tight_layout()
plt.show()
```

Tabela Consolidada de Resultados:

	Accuracy	Precision	Recall	F1
Perceptron	0.6300667	0.3629787	0.0565217	0.0105081
DT Default	0.7232222	0.3872202	0.4025015	0.3939994
DT Otimizada	0.7818889	0.5050006	0.5470404	0.5291969
Random Forest	0.7829267	0.5576172	0.3875017	0.4572426

Conclusão: Qual modelo levar para a produção?

1. A Evolução Técnica Nossa jornada deixou claro que o problema de crédito não se resolve com fórmulas simples. O Perceptron bateu no teto logo de cara por ser apenas uma linha reta. Quando mudamos e abordagem para falar os dados com a Árvore de Decisão, o ganho preditivo foi inevitável. No fim, o **Random Forest** se consolidou como o melhor algoritmo puro, entregando estabilidade e assertividade.

2. O Dor do Bolso (Analisando as Métricas de Confusão) No mundo real do crédito, não basta olhar a "Acurácia". Precisamos focar nos erros.

- **O Falso Negativo:** o cara que o modelo diz que é bom, mas ele calotea e o erro que causa rombo no caixa do banco.
- **O Falso Positivo:** (negar crédito a um bom cliente) faz o banco deixar de ganhar dinheiro. Como usamos `class_weight='balanced'` nas configurações, forçamos os nossos modelos de árvore a prestarem muita atenção nos calateiros, melhorando significativamente a taxa de acerto (Recall) na classe de inadimplentes.

3. O Trade-Off Final: Explicar vs. Lucrar Qual modelo vai colocamos em produção amanhã? Depende do cenário corporativo:

- **Se a regra do jogo for compliance e clareza:** Se o Banco Central ou a área jurídica exigir que a gente consiga explicar o motivo exato de cada recusa de crédito para o consumidor, ou escolhemos a **Árvore de Decisão Otimizada**. O custo de performance e menos vale a pena para ter um modelo transparente e auditável ("caixa-branca").
- **Se a regra do jogo for reduzir perdas e otimizar margem ser longa escala:** Se o foco do projeto for puramente atrelar o risco financeiro global do portfólio, eu vou com o **Random Forest**. O custo de processamento a mais para treinar dezenas de árvores (custo computacional) é ofto se compensado ao diminuir saíra por ele erros menores. Ele generaliza melhor, e o fato de ser uma "caixa-preta" é um trade-off que se paga em eficiência.